

Faculty of Information Technology, Monash University

COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

This material has been reproduced and communicated to you by or on behalf of Monash University pursuant to Part VB of the Copyright Act 1968 (the Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice

FIT2004: Algorithms and Data Structures

Week 2: Analysis of Algorithms and Sorting

Outline

- Proving correctness of algorithms
- Comparison-based sorting algorithms
 - Selection sort
 - Lower bound for comparison-based sorting
- Non-comparison sorting algorithms
 - Counting Sort
 - Radix Sort

Algorithmic Analysis

- In algorithmic analysis, one is interested in (at least) two things:
 - An algorithm's correctness.
 - The amount of resources used by the algorithm.
- We will now look at how to prove correctness.

Proving Correctness of Algorithms

- Commonly, we write programs and then **test** them.
- Testing detects inputs for which the program has incorrect output.
 - Can we test all possible inputs?
- There are infinitely many possible inputs (for most programs) so we cannot test them all, therefore...
- Testing cannot guarantee that the program is always correct!
- **Logic can** prove that a program is always correct. This is usually achieved in two parts:
 1. Show that the program **always** terminates;
 2. Show that a program produces **correct results** when it terminates.

Finding Minimum Value

```
//Find minimum value in an unsorted array of N>0 elements
min = array[1] // index starts at 1
index = 2

while index <= N

    if array[index] < min
        min = array[index]
    index = index + 1

return min
```

Does it Always Terminate?

```
//Find minimum value in an unsorted array of N>0 elements
min = array[1]
index = 2

while index <= N

    if array[index] < min
        min = array[index]
    index = index + 1

return min
```

Correct Result at Termination?

//Find minimum value in an unsorted array of $N > 0$ elements

```
min = array[1]
```

```
index = 2
```

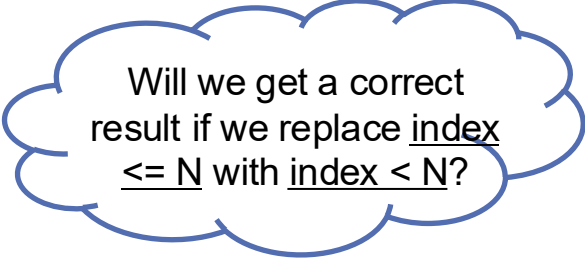
```
while index <= N
```

```
    if array[index] < min
```

```
        min = array[index]
```

```
    index = index + 1
```

```
return min
```



Will we get a correct result if we replace index <= N with index < N?

Correctness using Loop Invariant

- A loop invariant is a statement about an algorithm's loop
- Invariant should be true at three points:
 1. Before the loop starts (initialisation)
 2. During each loop (maintenance)
 3. After the loop terminates (termination)

Correctness using Loop Invariant

```
min = array[1]
index = 2
//LI: min equals the minimum value in array[1 .. index - 1]
while index <= N
    if array[index] < min
        min = array[index]
    index = index + 1
return min
```

Initialisation

The easiest way to do this is often to consider the invariant just **before** the loop condition is checked.

- `min = array[1]`
- `index = 2`
- `array[1 .. index - 1] = array[1...1]`

As required

Correctness using Loop Invariant

```
min = array[1]
index = 2
//LI: min equals the minimum value in array[1 .. index - 1]
while index <= N
    if array[index] < min
        min = array[index]
    index = index + 1
return min
```

Maintenance

- If LI was true when `index` was `k`, is it still true when `index` is `k+1`?
- To prove this, examine the code in the body of the loop and reason about the invariant.
- Remember to use the assumption that LI was true when `index` was `k`

Correctness using Loop Invariant

```
min = array[1]
index = 2
//LI: min equals the minimum value in array[1 ... index - 1]
while index <= N
    if array[index] < min
        min = array[index]
    index = index + 1
return min
```

Termination

- We have shown that LI is true for all values of `index` (in the previous step)
- Set `index` to the value which causes termination ($N+1$)
- Check if the statement of LI with this `index` value is what we want
- //LI: min equals the minimum value in array[1 ... N] as required

A Harder Problem

lo = 1 (we are 1 indexed here)

hi = N

```
while ( lo <= hi )
    mid = floor( (lo+hi)//2 )
    if target > array[mid]
        lo=mid+1
    else if target < array[mid]
        hi=mid-1
    else
        return mid
```

Return False

Binary search: Finds **an** instance of target, if it exists

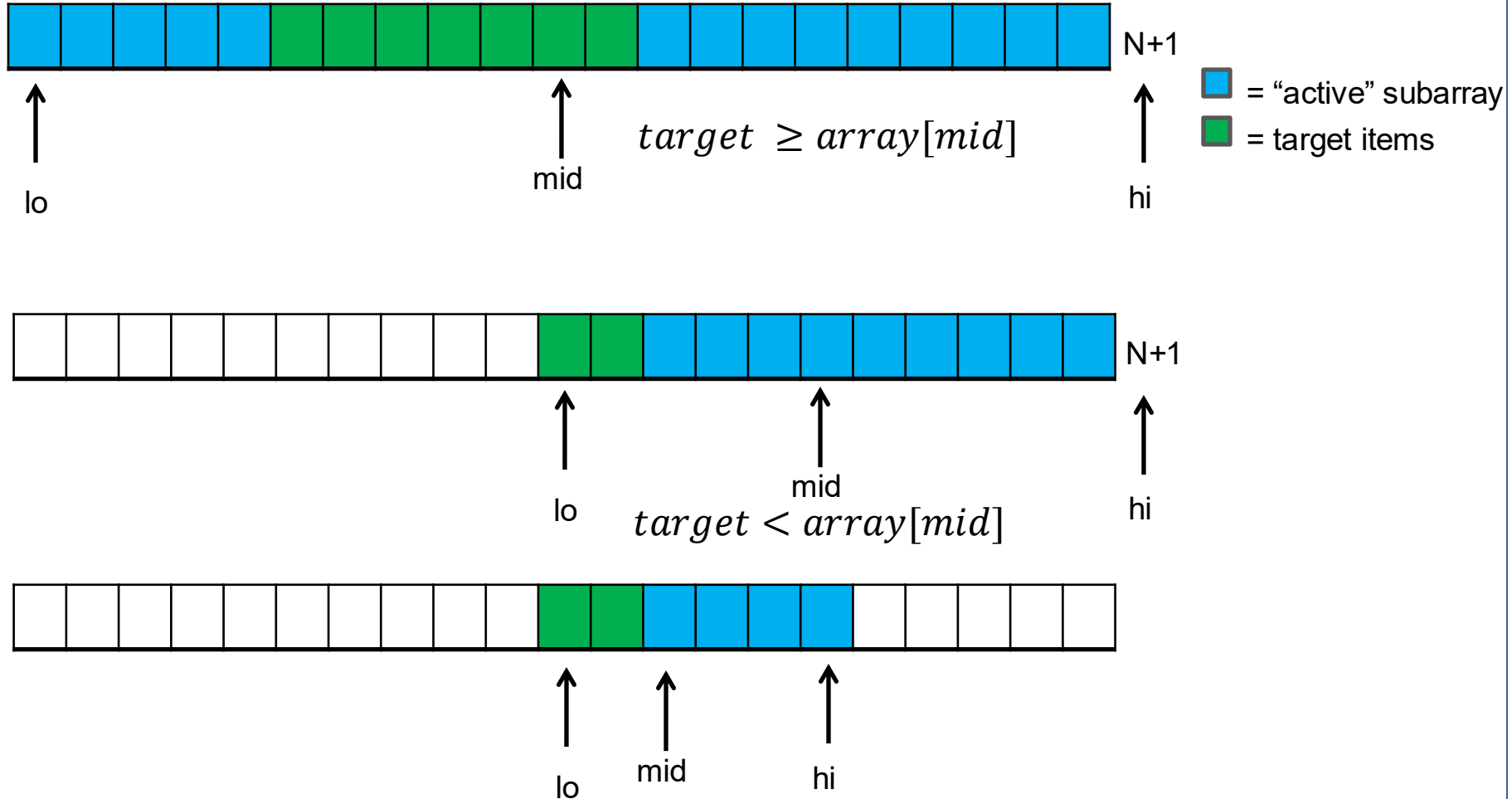
Always terminates (why?)

What if there is duplicate occurrences of target?

We don't know which one it will find

Let's solve the problem of finding the **rightmost** instance of target

Binary Search: Rightmost Instance



Binary Search: Rightmost Instance

- We want to move **lo** when the rightmost instance of target is to the right of mid $\rightarrow t > L[mid]$
- We want to move **hi** when the rightmost instance of target is to the left of mid $\rightarrow t < L[mid]$
- We want to move **lo** when a target **is at** mid (since we are looking for the rightmost) $\rightarrow t == L[mid]$
- This suggest we only have 2 cases:
 - $t < L[mid] \rightarrow$ move hi to mid
 - $t \geq L[mid] \rightarrow$ move lo to mid

Binary Search Invariant

5	10	15	20	25	30	35	40	45	50
1	2	3	4	5	6	7	8	9	10

- Invariant must relate to relevant variables (`lo`, `hi`, `array`).
- At termination, the invariant should tell us that the algorithm works.
- Example: min is the minimum of `array[1...N]` .



<https://flux.qa/F7CWW9>

Binary Search Invariant

- One possible invariant:
 - Rightmost instance is between lo (inclusive) and hi (exclusive)
- Is this true at the start?
 - Yes, if it exists...
- Invariant: if target is in the list, the rightmost instance is in $list[lo...hi-1]$

Design from Invariant

Invariant: If key is in L, it is in $L[lo \dots hi-1]$

```
lo = 1
```

```
hi = N+1
```

```
while ( lo ? hi )
```

When does the “active” section of the list have one element?

i.e. when is $L[lo \dots hi-1]$ a single item?

When $lo = hi-1$ (or $lo \geq hi-1$)

The while condition should be the opposite

i.e. we should continue the loop as long as there the active section has more than one item

While $lo < hi-1$

Design from Invariant

Invariant: If key is in L, it is in $L[lo \dots hi-1]$

```
lo = 1
```

```
hi = N+1
```

```
while ( lo < hi - 1 )
```

When does the “active” section of the list have one element?

i.e. when is $L[lo \dots hi-1]$ a single item?

When $lo = hi-1$ (or $lo \geq hi-1$)

The while condition should be the opposite

i.e. we should continue the loop as long as there the active section has more than one item

While $lo < hi-1$

Design from Invariant

Invariant: If key is in L, it is in $L[lo \dots hi-1]$

```
lo = 1
hi = N+1
while ( lo < hi - 1 )
    mid =  $\lfloor (lo + hi) / 2 \rfloor$ 
    if  $t \geq L[mid]$ 
```

Where should we move lo to?

We know that the element we are looking for is **at or to the right of** mid

We should set $lo=mid$, since lo is inclusive

Design from Invariant

Invariant: If key is in L, it is in $L[lo \dots hi-1]$

```
lo = 1
hi = N+1
while ( lo < hi - 1 )
    mid =  $\lfloor (lo + hi) / 2 \rfloor$ 
    if  $t \geq L[mid]$ 
        lo = mid
```

Where should we move lo to?

We know that the element we are looking for is **at or to the right of** mid

We should set $lo=mid$, since lo is inclusive

Design from Invariant

Invariant: If key is in L, it is in $L[lo \dots hi-1]$

```
lo = 1
hi = N+1
while ( lo < hi - 1 )
    mid =  $\lfloor (lo + hi) / 2 \rfloor$ 
    if  $t \geq L[mid]$ 
        lo = mid
    if  $t < L[mid]$ 
        hi = mid
```

Where should we move hi to?

Remember, $L[hi]$ is excluded from the active range

We can move hi to mid, since we know $L[mid]$ is not what we are looking for

Design from Invariant

Invariant: If key is in L, it is in $L[lo \dots hi-1]$

```
lo = 1
hi = N+1
while ( lo < hi - 1 )
    mid =  $\lfloor (lo + hi) / 2 \rfloor$ 
    if  $t \geq L[mid]$ 
        lo = mid
    if  $t < L[mid]$ 
        hi = mid
```

When the loop ends, $lo = hi-1$.

The only element in the active range is $L[lo]$

This means that either $L[lo]$ is what we are looking for, or it isn't

Design from Invariant

Invariant: If key is in L, it is in $L[lo \dots hi-1]$

```
lo = 1
hi = N+1
while ( lo < hi - 1 )
    mid =  $\lfloor (lo + hi) / 2 \rfloor$ 
    if  $t \geq L[mid]$ 
        lo = mid
    if  $t < L[mid]$ 
        hi = mid
if  $L[lo] == t$ 
    return lo
return False
```

When the loop ends, $lo = hi-1$.

The only element in the active range is $L[lo]$

This means that either $L[lo]$ is what we are looking for, or it isn't

Note that this algorithm improves over the “standard” 3-check algorithm since it does less work each iteration

Design from Invariant

Invariant: If key is in L, it is in $L[lo \dots hi-1]$

45	50
----	----

1 2 t= 50
lo hi mid =??

lo = 1

hi = N+1

while (lo < hi - 1)

 mid = $\lfloor (lo + hi) / 2 \rfloor$

if t $\geq$ L[mid]

 lo = mid

if t < L[mid]

 hi = mid

if L[lo] == t

return lo

return False

Try to think about what would happen if we **included** hi (i.e. we try the invariant

If key is in L, it is in $L[lo \dots hi]$

The issue is in the while condition

It would need to be while (lo < hi)

What happens when lo = hi - 1 and t $\geq$ L[mid]? Does it always terminate?

Algorithm for Binary Search

Invariant: If key is in L, it is in $L[lo \dots hi-1]$

```
BinarySearch(L[1..n], t)
  lo = 1
  hi = N+1
  while ( lo < hi - 1 )
    mid =  $\lfloor (lo + hi) / 2 \rfloor$ 
    if  $t \geq L[mid]$ 
      lo = mid
    if  $t < L[mid]$ 
      hi = mid
  if  $L[lo] == t$ 
    return lo
  return False
```

Is this algorithm correct?

To prove correctness, we need to show that

1. it **always** terminates, and
2. it returns correct result when it terminates

Correctness using Loop Invariant

Invariant: If key is in L, it is in $L[lo \dots hi-1]$

```
BinarySearch(L[1..n], t)
  lo = 1
  hi = N+1
  while ( lo < hi - 1 )
    mid =  $\lfloor (lo + hi) / 2 \rfloor$ 
    if  $t \geq L[mid]$ 
      lo = mid
    if  $t < L[mid]$ 
      hi = mid
  if  $L[lo] == t$ 
    return lo
  return False
```

Always terminates

- Algorithm terminates when $lo \geq hi - 1$
- Because of the while condition, each iteration, after mid is calculated, $lo < mid$, $mid < hi$ (why?)
- When we set lo to mid, lo must therefore increase by at least 1
- Similarly, when we set hi to mid, hi must decrease by at least 1
- So every iteration, either lo increases or hi decreases.
- Eventually, $lo \geq hi - 1$

Correctness using Loop Invariant

Invariant: If key is in L, it is in $L[lo \dots hi-1]$

```
BinarySearch(L[1..n], t)
  lo = 1
  hi = N+1
  while ( lo < hi - 1 )
    mid =  $\lfloor (lo + hi) / 2 \rfloor$ 
    if  $t \geq L[mid]$ 
      lo = mid
    if  $t < L[mid]$ 
      hi = mid
  if  $L[lo] == t$ 
    return lo
  return False
```

Initialisation

- If rightmost target is in L, then
target is in $L[lo \dots hi-1] =$
 $L[1 \dots N] = L$

As required

Correctness using Loop Invariant

Invariant: If key is in L, it is in $L[lo \dots hi-1]$

```
BinarySearch(L[1..n], t)
  lo = 1
  hi = N+1
  while ( lo < hi - 1 )
    mid =  $\lfloor (lo + hi) / 2 \rfloor$ 
    if  $t \geq L[mid]$ 
      lo = mid
    if  $t < L[mid]$ 
      hi = mid
  if  $L[lo] == t$ 
    return lo
  return False
```

Maintenance

- Assume target in $L[lo \dots hi-1]$ before some loop iteration
- If $t \geq L[mid]$, then it must be located in the range $L[mid \dots hi-1]$, so we should set lo to mid
- If $t < L[mid]$, then it must be in the range $L[lo \dots mid-1]$, so we should set hi to mid (since $L[hi]$ is excluded)
- Since this is exactly what we do, at the start of the next iteration the invariant is still true

Correctness using Loop Invariant

Invariant: If key is in L, it is in $L[lo...hi-1]$

```
BinarySearch(L[1..n], t)
  lo = 1
  hi = N+1
  while ( lo < hi - 1 )
    mid =  $\lfloor (lo + hi) / 2 \rfloor$ 
    if  $t \geq L[mid]$ 
      lo = mid
    if  $t < L[mid]$ 
      hi = mid
  if  $L[lo] == t$ 
    return lo
  return False
```

Termination

- $lo == hi - 1$. Since the invariant has been correctly maintained...
- Rightmost target must be in $L[lo...hi-1]$, or it is not in the list
- $L[lo...hi-1]$ is a single element, $L[lo]$.
- If the element is found at $L[lo]$, then return that index
- Otherwise return False

Outline

- Proving correctness of algorithms
- **Comparison-based sorting algorithms**
 - Selection sort
 - Lower bound for comparison-based sorting
- **Non-comparison sorting algorithms**
 - Counting Sort
 - Radix Sort

Comparison-based Sorting

- Comparison-based sorting algorithms sort the input array by comparing the items with each other:
 - Selection Sort
 - Insertion Sort
 - Quick Sort (to be analysed next week)
 - Merge Sort
 - Heap Sort
 - ...
- The algorithms that do not require comparing items with each other are called non-comparison sorting algorithms. E.g., Counting sort, radix sort, bucket sort etc.



Comparison Cost

- Typically, we assume that comparing two elements takes $O(1)$, e.g., `array[i] <= array[j]`. This is not necessarily true.
- **String Comparison:** The worst-case cost of comparing two strings is $O(L)$ where L is the number of characters in the smaller string. E.g.,
 - “Welcome to Faculty of IT” <= “Welcome to FIT2004” ??
 - We compare strings character by character (from left to right) until the two characters are different – all green letters are compared in above example
- **Number Comparison:** Generally we assume that numbers are machine-word sized (i.e. fit in a register) and so comparison is $O(1)$. In theory, for very large numbers, comparison would be $O(\text{digits})$.

Comparison Cost

- Typically assume comparison cost is $O(1)$ (small values)
- Sometimes comparison cost is critical: genome sequences may have millions of characters – expensive comparison.
- The cost of comparison-based sorting is often taken as in terms of # of comparisons, e.g., # of comparisons in merge sort is $O(N \log N)$.
- This ignores comparison cost (which is mostly fine).

In summary:

In this unit we will generally assume that

- String comparison is $O(c)$, where c is the number of characters which get compared
- Numerical comparison is $O(1)$ unless otherwise specified

Stable Sorting Algorithms

- A sorting algorithm is called stable if it maintains the relative ordering of elements that have equal keys.

Input is sorted by names

Input

Marks	80	75	70	90	85	75
Name	Alice	Bill	Don	Geoff	Leo	Maria

Sort on Marks using a stable algorithm



Output

Marks	70	75	75	80	85	90
Name	Don	Bill	Maria	Alice	Leo	Geoff

Note: Output is sorted on marks then names.

Unstable sorting cannot guarantee this (e.g., Maria may appear before Bill)

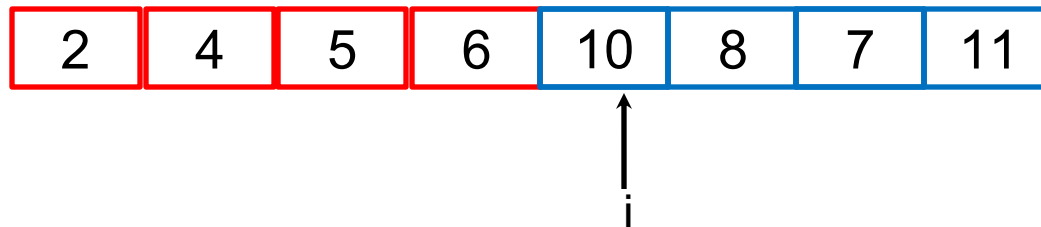
Outline

- Proving correctness of algorithms
- Comparison-based sorting algorithms
 - Selection sort
 - Lower bound for comparison-based sorting
- Non-comparison sorting algorithms
 - Counting Sort
 - Radix Sort

Selection Sort (Correctness)

Sort an array (denoted as arr) in ascending order

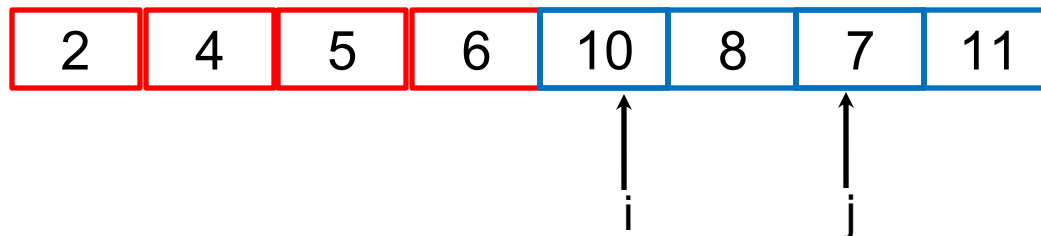
```
for(i = 1; i < N; i++) {  
    j = index of minimum element in arr[i ... N]  
    swap (arr[i],arr[j])  
}
```



Selection Sort (Correctness)

Sort an array (denoted as arr) in ascending order

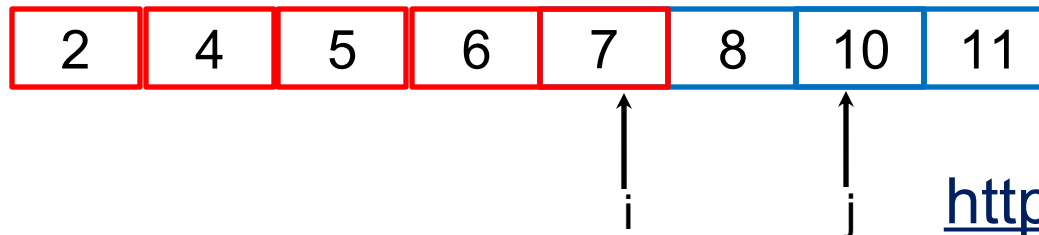
```
for(i = 1; i < N; i++) {  
    j = index of minimum element in arr[i ... N]  
    swap (arr[i],arr[j])  
}
```



Selection Sort (Correctness)

Sort an array (denoted as arr) in ascending order

```
for(i = 1; i < N; i++) {  
    j = index of minimum element in arr[i ... N]  
    swap (arr[i],arr[j])  
}
```

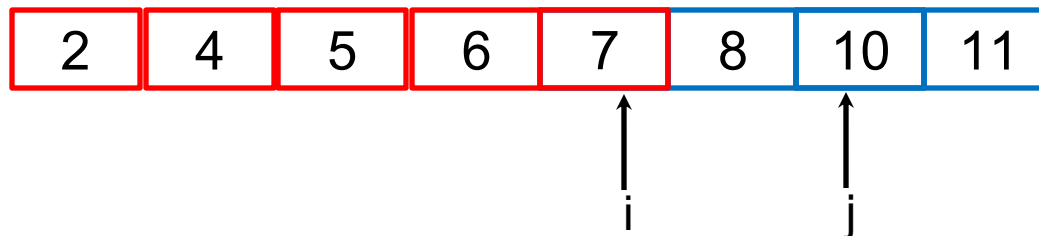


<https://flux.qa/F7CWW9>

Selection Sort (Correctness)

Sort an array (denoted as arr) in ascending order

```
// LI: arr[1 ... i-1] is sorted AND arr[1 ... i-1] <= arr[i ... N]
for(i = 1; i < N; i++) {
    j = index of minimum element in arr[i ... N]
    swap (arr[i], arr[j])
}
// i=N when the loop terminates
// LI: arr[1 ... N-1] is sorted AND arr[1 ... N-1] <= arr[N]
```



Selection Sort (Correctness)

Sort an array (denoted as `arr`) in ascending order

```
// LI: arr[1 ... i-1] is sorted AND arr[1 ... i-1] <= arr[i ... N]
for(i = 1; i < N; i++) {
    j = index of minimum element in arr[i ... N]
    swap (arr[i], arr[j])
}
// i=N when the loop terminates
// LI: arr[1 ... N-1] is sorted AND arr[1 ... N-1] <= arr[N]
```

Could we use a weaker loop invariant, e.g.,

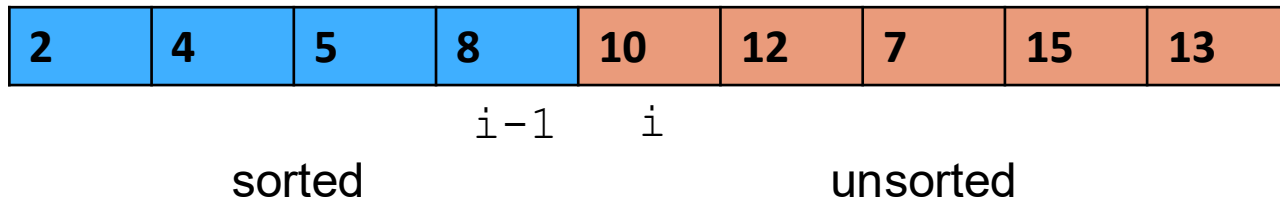
```
// LI: arr[1 ... i-1] is sorted (That is Insertion Sort)
```

Selection Sort (Correctness)

Could we use a weaker loop invariant, e.g.,
// LI: `arr[1 .. i-1]` **is** sorted (That is Insertion Sort)

While doing the proof, we need to show that if the LI is true at iteration k , it is true at iteration $k+1$

```
j = index of minimum element in arr[i .. N]
swap (arr[i], arr[j])
```

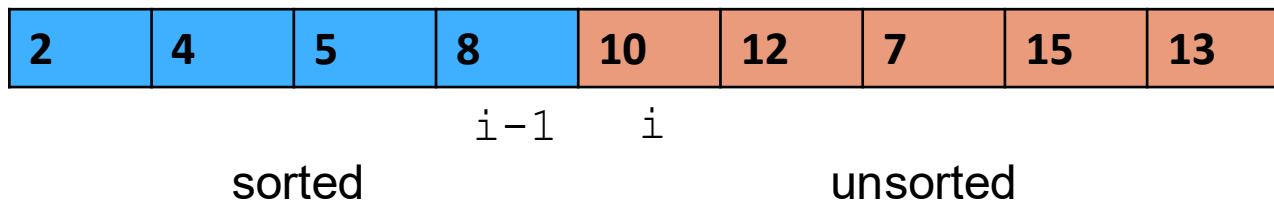


Selection Sort (Correctness)

Could we use a weaker loop invariant, e.g.,
// LI: `arr[1 .. i-1]` **is** sorted (That is Insertion Sort)

While doing the proof, we need to show that if the LI is true at iteration k , it is true at iteration $k+1$

```
j = index of minimum element in arr[i .. N]
swap (arr[i], arr[j])
```



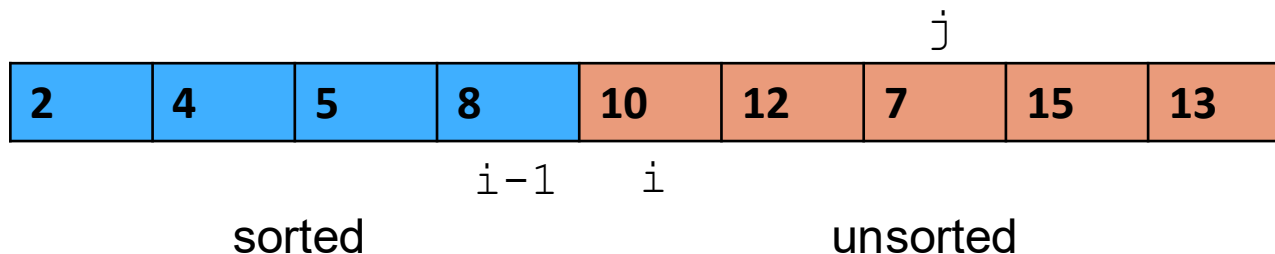
The invariant currently holds

Selection Sort (Correctness)

Could we use a weaker loop invariant, e.g.,
// LI: `arr[1 .. i-1]` **is** sorted (That is Insertion Sort)

While doing the proof, we need to show that if the LI is true at iteration k , it is true at iteration $k+1$

j = index of minimum element in `arr[i .. N]`
swap (`arr[i]`, `arr[j]`)



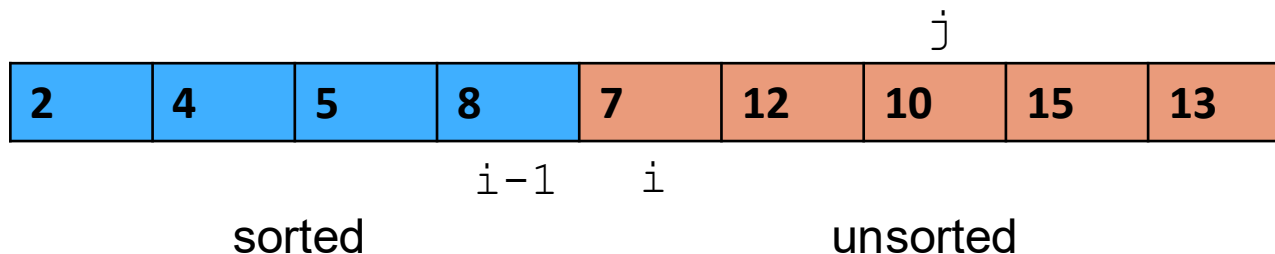
The invariant currently holds

Selection Sort (Correctness)

Could we use a weaker loop invariant, e.g.,
// LI: `arr[1 .. i-1]` **is** sorted (That is Insertion Sort)

While doing the proof, we need to show that if the LI is true at iteration k , it is true at iteration $k+1$

```
j = index of minimum element in arr[i .. N]
swap (arr[i], arr[j])
```



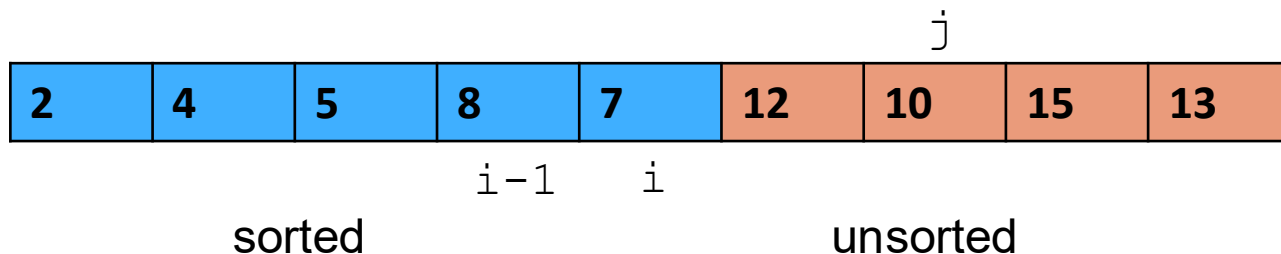
The invariant currently holds

Selection Sort (Correctness)

Could we use a weaker loop invariant, e.g.,
// LI: `arr[1 .. i-1]` **is** sorted (That is Insertion Sort)

While doing the proof, we need to show that if the LI is true at iteration k , it is true at iteration $k+1$

```
j = index of minimum element in arr[i .. N]
swap (arr[i], arr[j])
```



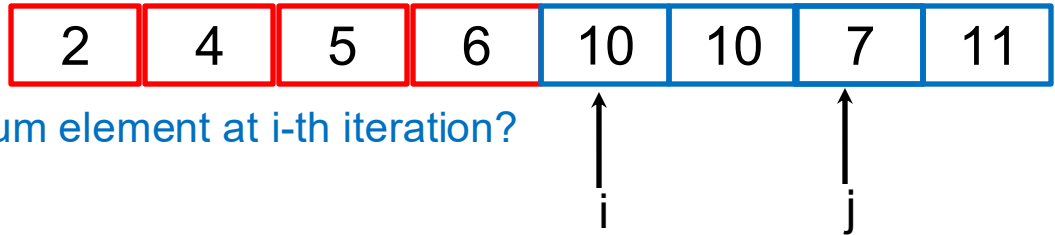
The invariant should still be true... but it is not

Selection Sort Analysis

```
for(i = 1; i < N; i++) {  
    j = index of minimum element in arr[i ... N]  
    swap (arr[i], arr[j])  
}
```

Time Complexity?

- Worst-case
 - Complexity of finding minimum element at i-th iteration?
 - Total complexity?
- Best-case



Space Complexity?

Auxiliary Space Complexity?

- Selection Sort is an in-place algorithm!

Is selection sort stable?

Outline

- Proving correctness of algorithms
- **Comparison-based sorting algorithms**
 - Selection sort
 - **Lower bound for comparison-based sorting**
- **Non-comparison sorting algorithms**
 - Counting Sort
 - Radix Sort

Summary of Comparison-based Sorting Algorithms

	Best	Worst	Average	Stable?	In-place?
Selection Sort	$O(N^2)$	$O(N^2)$	$O(N^2)$	No	Yes
Insertion Sort	$O(N)$	$O(N^2)$	$O(N^2)$	Yes	Yes
Heap Sort	$O(N)$	$O(N \log N)$	$O(N \log N)$	No	Yes
Merge Sort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	Yes	No

- Is it possible to develop a sorting algorithm with worst-case time complexity better than $O(N \log N)$?

Lower Bound Complexity

- **Lower bound complexity** for a problem is the lowest possible complexity any algorithm (known or unknown) can achieve to solve the problem.
 - It is important because it gives a theoretical bound on what is best possible.
 - Unless stated otherwise, lower bound is for the worst-case complexity of the algorithm.
- What is the lower bound complexity of finding the minimum element in an array of N elements
 - **Ans:** $\Omega(N)$: We (at least) need to look at every element.
- Since the finding minimum algorithm we saw has $O(N)$ worst-case complexity, it is the best possible algorithm (i.e., optimal) in terms of time complexity.

Lower Bound Complexity

- **Lower bound complexity** for a problem is the lowest possible complexity any algorithm (known or unknown) can achieve to solve the problem.
 - It is important because it gives a theoretical bound on what is best possible.
 - Unless stated otherwise, lower bound is for the worst-case complexity of the algorithm.
- What is the lower bound complexity for sorting?
 - For comparison-based algorithm, lower bound complexity is $\Omega(N \log N)$.
 - Read the lecture notes to see why the lower bound is $\Omega(N \log N)$ -- the proof is not examinable.
- Next, we discuss two non-comparison sorting algorithms that do sorting in less than $O(N \log N)$.

Outline

- Proving correctness of algorithms
- Comparison-based sorting algorithms
 - Selection sort
 - Lower bound for comparison-based sorting
- **Non-comparison sorting algorithms**
 - Counting Sort
 - Radix Sort

Intuition

- Given an array containing a permutation of the numbers $1 \dots N$, sort them.

```
for (i=1, i <= N, i++)  
    print(i)
```

- Given an array containing a subset of the numbers $1 \dots N$, sort them.

Counting Sort

- For example, sorting positive integers.
- Determine the maximum value in the input.
- Create an array “count” of that size.

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

count

1	0
2	0
3	0
4	0
5	0
6	0
7	0
8	0

Counting Sort

- Iterate through the input and count the number of times each value occurs.
- Do this by incrementing the corresponding position in “count”.
- If we see a 3, increment *count*[3].

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

count

1	0
2	0
3	0
4	0
5	0
6	0
7	0
8	0

Counting Sort

- Iterate through the input and count the number of times each value occurs.
- Do this by incrementing the corresponding position in “count”.
- If we see a 3, increment *count*[3].

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

count

1	0
2	0
3	1
4	0
5	0
6	0
7	0
8	0

Counting Sort

- Iterate through the input and count the number of times each value occurs.
- Do this by incrementing the corresponding position in “count”.
- If we see a 3, increment *count*[3].

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

count

1	1
2	0
3	1
4	0
5	0
6	0
7	0
8	0

Counting Sort

- Iterate through the input and count the number of times each value occurs.
- Do this by incrementing the corresponding position in “count”.
- If we see a 3, increment *count*[3].

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

count

1	1
2	0
3	2
4	0
5	0
6	0
7	0
8	0

Counting Sort

- Iterate through the input and count the number of times each value occurs.
- Do this by incrementing the corresponding position in “count”.
- If we see a 3, increment *count*[3].

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

count

1	1
2	0
3	2
4	0
5	0
6	0
7	1
8	0

Counting Sort

- Iterate through the input and count the number of times each value occurs.
- Do this by incrementing the corresponding position in “count”.
- If we see a 3, increment *count*[3].

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

count

1	1
2	0
3	2
4	0
5	1
6	0
7	1
8	0

Counting Sort

- Iterate through the input and count the number of times each value occurs.
- Do this by incrementing the corresponding position in “count”.
- If we see a 3, increment *count*[3].

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

count

1	1
2	0
3	3
4	0
5	1
6	0
7	1
8	0

Counting Sort

- Iterate through the input and count the number of times each value occurs.
- Do this by incrementing the corresponding position in “count”.
- If we see a 3, increment *count*[3].

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	0

Counting Sort

- Iterate through the input and count the number of times each value occurs.
- Do this by incrementing the corresponding position in “count”.
- If we see a 3, increment *count*[3].

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

Counting Sort

- Create “output”.
- For each position in count:
 - Append $count[i]$ copies of the value i to output.
 - So if $count[7] = 2$, then append 2 copies of 7.

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Output

--	--	--	--	--	--	--	--

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

Counting Sort

- Create “output”.
- For each position in count:
 - Append $count[i]$ copies of the value i to output.
 - So if $count[7] = 2$, then append 2 copies of 7.

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Output

1							
---	--	--	--	--	--	--	--

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

Counting Sort

- Create “output”.
- For each position in count:
 - Append $count[i]$ copies of the value i to output.
 - So if $count[7] = 2$, then append 2 copies of 7.

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Output

1							
---	--	--	--	--	--	--	--

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

Counting Sort

- Create “output”.
- For each position in count:
 - Append $count[i]$ copies of the value i to output.
 - So if $count[7] = 2$, then append 2 copies of 7.

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Output

1	3	3	3				
---	---	---	---	--	--	--	--

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

Counting Sort

- Create “output”.
- For each position in count:
 - Append $count[i]$ copies of the value i to output.
 - So if $count[7] = 2$, then append 2 copies of 7.

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Output

1	3	3	3				
---	---	---	---	--	--	--	--

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

Counting Sort

- Create “output”.
- For each position in count:
 - Append $count[i]$ copies of the value i to output.
 - So if $count[7] = 2$, then append 2 copies of 7.

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Output

1	3	3	3	5			
---	---	---	---	---	--	--	--

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

Counting Sort

- Create “output”.
- For each position in count:
 - Append $count[i]$ copies of the value i to output.
 - So if $count[7] = 2$, then append 2 copies of 7.

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Output

1	3	3	3	5			
---	---	---	---	---	--	--	--

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

Counting Sort

- Create “output”.
- For each position in count:
 - Append $count[i]$ copies of the value i to output.
 - So if $count[7] = 2$, then append 2 copies of 7.

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Output

1	3	3	3	5	7	7	
---	---	---	---	---	---	---	--

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

Counting Sort

- Create “output”.
- For each position in count:
 - Append $count[i]$ copies of the value i to output.
 - So if $count[7] = 2$, then append 2 copies of 7.

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Output

1	3	3	3	5	7	7	8
---	---	---	---	---	---	---	---

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

Analysis of Counting Sort

- Create “count” array of size max, where max is the maximum value in the input.
- For each value in “Input”:
 - `count[value] += 1`
- Output = empty
- For $x=1$ to $\text{len}(\text{count})$:
 - `NumOfOccurrences = count[x]`
 - Append x to Output `NumOfOccurrences` times

Let N be the size of Input array and U be the domain size (e.g., max), i.e., U is the size of count array.

Time Complexity:

- $O(N+U)$ – worst-case, best-case, average-case all are the same.

Space Complexity:

- $O(N+U)$

Analysis of Counting Sort

- Create “count” array of size max, where max is the maximum value in the input.
- For each value in “Input”:
 - `count[value] += 1`
- Output = empty
- For x=1 to len(count):
 - `NumOfOccurrences = count[x]`
 - Append x to Output NumOfOccurrences times

Is counting sort stable?

No, because it counts the values but does not distinguish between them. In fact, it loses any data associated with the values, so it is much worse than unstable. **Let's fix this!**

Input	(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
Output	1	3	3	3	5	7	7	8

Stable Counting Sort

- To fix the two problems of stability and losing data, we need a smart idea.

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Output

(1,p)	(3,a)	(3,c)	(3,b)	(5,g)	(7,f)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

- What information would you need to correctly place the data with key “5” in the output?

Stable Counting Sort

Input	(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- $\text{count}[\text{key}] += 1$

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

Stable Counting Sort

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- $\text{count}[\text{key}] += 1$

Construct position:

- Initialise first position as a 1

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

position

1	1
2	0
3	0
4	0
5	0
6	0
7	0
8	0

Stable Counting Sort

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- $\text{count}[\text{key}] += 1$

Construct position:

- Initialise first position as a 1
- $\text{position}[i] = \text{position}[i-1] + \text{count}[i-1]$

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

position

1	1
2	2
3	0
4	0
5	0
6	0
7	0
8	0

Stable Counting Sort

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- $\text{count}[\text{key}] += 1$

Construct position:

- Initialise first position as a 1
- $\text{position}[i] = \text{position}[i-1] + \text{count}[i-1]$

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

position

1	1
2	2
3	2
4	0
5	0
6	0
7	0
8	0

Stable Counting Sort

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- $\text{count}[\text{key}] += 1$

Construct position:

- Initialise first position as a 1
- $\text{position}[i] = \text{position}[i-1] + \text{count}[i-1]$

count		position	
1	1	1	1
2	0	2	2
3	3	3	2
4	0	4	5
5	1	5	0
6	0	6	0
7	2	7	0
8	1	8	0

Stable Counting Sort

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- $\text{count}[\text{key}] += 1$

Construct position:

- Initialise first position as a 1
- $\text{position}[i] = \text{position}[i-1] + \text{count}[i-1]$

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

position

1	1
2	2
3	2
4	5
5	5
6	0
7	0
8	0

Stable Counting Sort

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- $\text{count}[\text{key}] += 1$

Construct position:

- Initialise first position as a 1
- $\text{position}[i] = \text{position}[i-1] + \text{count}[i-1]$

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

position

1	1
2	2
3	2
4	5
5	5
6	6
7	0
8	0

Stable Counting Sort

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- $\text{count}[\text{key}] += 1$

Construct position:

- Initialise first position as a 1
- $\text{position}[i] = \text{position}[i-1] + \text{count}[i-1]$

count		position	
1	1	1	1
2	0	2	2
3	3	3	2
4	0	4	5
5	1	5	5
6	0	6	6
7	2	7	6
8	1	8	0

Stable Counting Sort

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- $\text{count}[\text{key}] += 1$

Construct position:

- Initialise first position as a 1
- $\text{position}[i] = \text{position}[i-1] + \text{count}[i-1]$

count		position	
1	1	1	1
2	0	2	2
3	3	3	2
4	0	4	5
5	1	5	5
6	0	6	6
7	2	7	6
8	1	8	8

Stable Counting Sort

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- $\text{count}[\text{key}] += 1$

Construct position:

- Initialise first position as a 1
- $\text{position}[i] = \text{position}[i-1] + \text{count}[i-1]$

Construct output

- Go through input, looking at each (key, val)
- Set $\text{output}[\text{position}[\text{key}]]$ to the (key, val) pair from input
- Increment $\text{position}[\text{key}]$

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

position

1	1
2	2
3	2
4	5
5	5
6	6
7	6
8	8

Output

1	2	3	4	5	6	7	8

Stable Counting Sort

Input (3,a) (1,p) (3,c) (7,f) (5,g) (3,b) (7,d) (8,w)

Construct count:

- For each key in input,
- $\text{count}[\text{key}] += 1$

Construct position:

- Initialise first position as a 1
- $\text{position}[i] = \text{position}[i-1] + \text{count}[i-1]$

Construct output

- Go through input, looking at each (key, val)
- Set $\text{output}[\text{position}[\text{key}]]$ to the (key, val) pair from input
- Increment $\text{position}[\text{key}]$

Output

	(3,a)						
1	2	3	4	5	6	7	8

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

position

1	1
2	2
3	2
4	5
5	5
6	6
7	6
8	8

Stable Counting Sort

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- $\text{count}[\text{key}] += 1$

Construct position:

- Initialise first position as a 1
- $\text{position}[i] = \text{position}[i-1] + \text{count}[i-1]$

Construct output

- Go through input, looking at each (key, val)
- Set $\text{output}[\text{position}[\text{key}]]$ to the (key, val) pair from input
- Increment $\text{position}[\text{key}]$

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

position

1	1
2	2
3	3
4	5
5	5
6	6
7	6
8	8

Output

	(3,a)						
--	-------	--	--	--	--	--	--

1 2 3 4 5 6 7 8

Stable Counting Sort

Input (3,a) (1,p) (3,c) (7,f) (5,g) (3,b) (7,d) (8,w)

Construct count:

- For each key in input,
- $\text{count}[\text{key}] += 1$

Construct position:

- Initialise first position as a 1
- $\text{position}[i] = \text{position}[i-1] + \text{count}[i-1]$

Construct output

- Go through input, looking at each (key, val)
- Set $\text{output}[\text{position}[\text{key}]]$ to the (key, val) pair from input
- Increment $\text{position}[\text{key}]$

Output (1,p) (3,a) 1 2 3 4 5 6 7 8

count		position	
1	1	1	1
2	0	2	2
3	3	3	3
4	0	4	5
5	1	5	5
6	0	6	6
7	2	7	6
8	1	8	8

Stable Counting Sort

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- $\text{count}[\text{key}] += 1$

Construct position:

- Initialise first position as a 1
- $\text{position}[i] = \text{position}[i-1] + \text{count}[i-1]$

Construct output

- Go through input, looking at each (key, val)
- Set $\text{output}[\text{position}[\text{key}]]$ to the (key, val) pair from input
- Increment $\text{position}[\text{key}]$

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

position

1	2
2	2
3	3
4	5
5	5
6	6
7	6
8	8

Output

(1,p)	(3,a)						
1	2	3	4	5	6	7	8

Stable Counting Sort

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- $\text{count}[\text{key}] += 1$

Construct position:

- Initialise first position as a 1
- $\text{position}[i] = \text{position}[i-1] + \text{count}[i-1]$

Construct output

- Go through input, looking at each (key, val)
- Set $\text{output}[\text{position}[\text{key}]]$ to the (key, val) pair from input
- Increment $\text{position}[\text{key}]$

Output

(1,p)	(3,a)	(3,c)					
1	2	3	4	5	6	7	8

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

position

1	2
2	2
3	3
4	5
5	5
6	6
7	6
8	8

Stable Counting Sort

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- $\text{count}[\text{key}] += 1$

Construct position:

- Initialise first position as a 1
- $\text{position}[i] = \text{position}[i-1] + \text{count}[i-1]$

Construct output

- Go through input, looking at each (key, val)
- Set $\text{output}[\text{position}[\text{key}]]$ to the (key, val) pair from input
- Increment $\text{position}[\text{key}]$

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

position

1	2
2	2
3	4
4	5
5	5
6	6
7	6
8	8

Output

(1,p)	(3,a)	(3,c)					
1	2	3	4	5	6	7	8

Stable Counting Sort

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- $\text{count}[\text{key}] += 1$

Construct position:

- Initialise first position as a 1
- $\text{position}[i] = \text{position}[i-1] + \text{count}[i-1]$

Construct output

- Go through input, looking at each (key, val)
- Set $\text{output}[\text{position}[\text{key}]]$ to the (key, val) pair from input
- Increment $\text{position}[\text{key}]$

Output

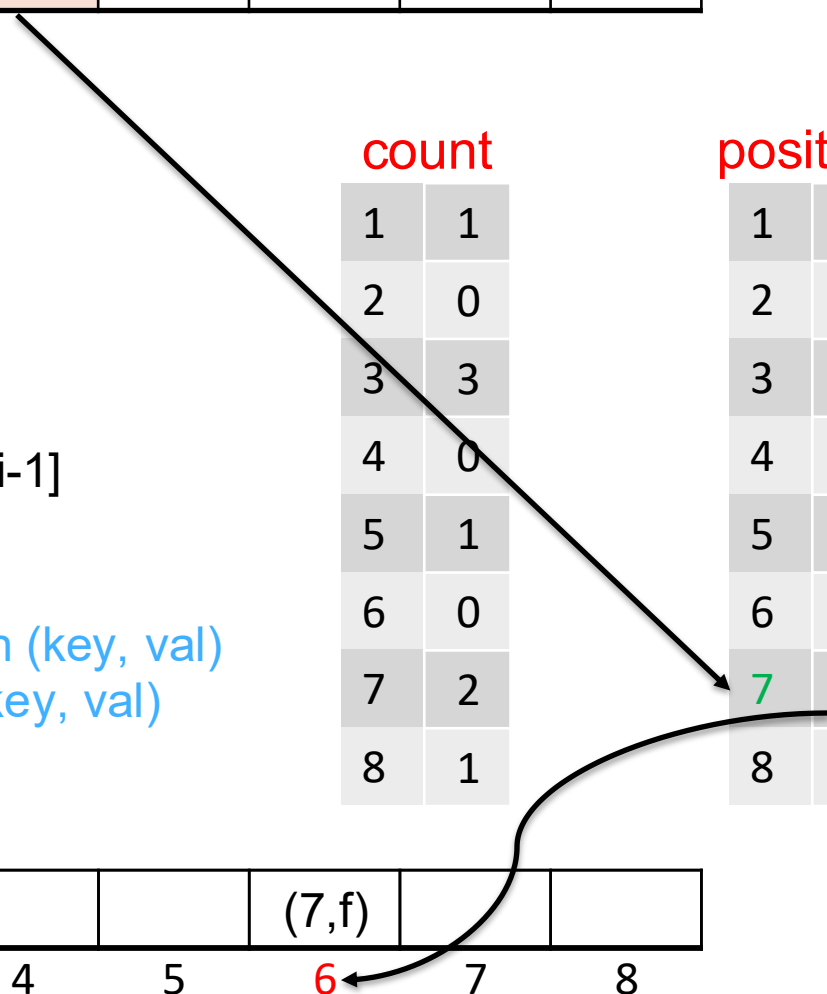
(1,p)	(3,a)	(3,c)			(7,f)		
1	2	3	4	5	6	7	8

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

position

1	2
2	2
3	4
4	5
5	5
6	6
7	6
8	8



Stable Counting Sort

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- count[key] += 1

Construct position:

- Initialise first position as a 1
- position[i] = position[i-1] + count[i-1]

Construct output

- Go through input, looking at each (key, val)
- Set output[position[key]] to the (key, val) pair from input
- Increment position[key]

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

position

1	2
2	2
3	4
4	5
5	5
6	6
7	7
8	8

Output

(1,p)	(3,a)	(3,c)			(7,f)		
1	2	3	4	5	6	7	8

Stable Counting Sort

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- $\text{count}[\text{key}] += 1$

Construct position:

- Initialise first position as a 1
- $\text{position}[i] = \text{position}[i-1] + \text{count}[i-1]$

Construct output

- Go through input, looking at each (key, val)
- Set $\text{output}[\text{position}[\text{key}]]$ to the (key, val) pair from input
- Increment $\text{position}[\text{key}]$

Output

(1,p)	(3,a)	(3,c)		(5,g)	(7,f)		
1	2	3	4	5	6	7	8

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

position

1	2
2	2
3	4
4	5
5	5
6	6
7	7
8	8

Stable Counting Sort

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- count[key] += 1

Construct position:

- Initialise first position as a 1
- position[i] = position[i-1] + count[i-1]

Construct output

- Go through input, looking at each (key, val)
- Set output[position[key]] to the (key, val) pair from input
- Increment position[key]

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

position

1	2
2	2
3	4
4	5
5	6
6	6
7	7
8	8

Output

(1,p)	(3,a)	(3,c)		(5,g)	(7,f)		
1	2	3	4	5	6	7	8

Stable Counting Sort

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- $\text{count}[\text{key}] += 1$

Construct position:

- Initialise first position as a 1
- $\text{position}[i] = \text{position}[i-1] + \text{count}[i-1]$

Construct output

- Go through input, looking at each (key, val)
- Set $\text{output}[\text{position}[\text{key}]]$ to the (key, val) pair from input
- Increment $\text{position}[\text{key}]$

Output

(1,p)	(3,a)	(3,c)	(3,b)	(5,g)	(7,f)		
1	2	3	4	5	6	7	8

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

position

1	2
2	2
3	4
4	5
5	6
6	6
7	7
8	8

Stable Counting Sort

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- count[key] += 1

Construct position:

- Initialise first position as a 1
- position[i] = position[i-1] + count[i-1]

Construct output

- Go through input, looking at each (key, val)
- Set output[position[key]] to the (key, val) pair from input
- Increment position[key]

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

position

1	2
2	2
3	5
4	5
5	6
6	6
7	7
8	8

Output

(1,p)	(3,a)	(3,c)	(3,b)	(5,g)	(7,f)		
1	2	3	4	5	6	7	8

Stable Counting Sort

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- $\text{count}[\text{key}] += 1$

Construct position:

- Initialise first position as a 1
- $\text{position}[i] = \text{position}[i-1] + \text{count}[i-1]$

Construct output

- Go through input, looking at each (key, val)
- Set $\text{output}[\text{position}[\text{key}]]$ to the (key, val) pair from input
- Increment $\text{position}[\text{key}]$

Output

(1,p)	(3,a)	(3,c)	(3,b)	(5,g)	(7,f)	(7,d)	
1	2	3	4	5	6	7	8

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

position

1	2
2	2
3	5
4	5
5	6
6	6
7	7
8	8

Stable Counting Sort

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- count[key] += 1

Construct position:

- Initialise first position as a 1
- position[i] = position[i-1] + count[i-1]

Construct output

- Go through input, looking at each (key, val)
- Set output[position[key]] to the (key, val) pair from input
- Increment position[key]

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

position

1	2
2	2
3	5
4	5
5	6
6	6
7	8
8	8

Output

(1,p)	(3,a)	(3,c)	(3,b)	(5,g)	(7,f)	(7,d)	
1	2	3	4	5	6	7	8

Stable Counting Sort

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- $\text{count}[\text{key}] += 1$

Construct position:

- Initialise first position as a 1
- $\text{position}[i] = \text{position}[i-1] + \text{count}[i-1]$

Construct output

- Go through input, looking at each (key, val)
- Set $\text{output}[\text{position}[\text{key}]]$ to the (key, val) pair from input
- Increment $\text{position}[\text{key}]$

Output

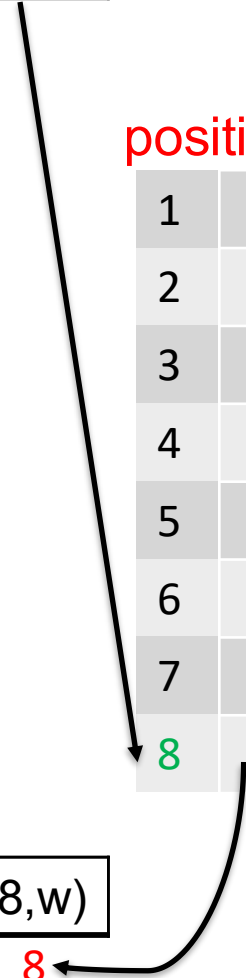
(1,p)	(3,a)	(3,c)	(3,b)	(5,g)	(7,f)	(7,d)	(8,w)
1	2	3	4	5	6	7	8

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

position

1	2
2	2
3	5
4	5
5	6
6	6
7	8
8	8



Stable Counting Sort

Input

(3,a)	(1,p)	(3,c)	(7,f)	(5,g)	(3,b)	(7,d)	(8,w)
-------	-------	-------	-------	-------	-------	-------	-------

Construct count:

- For each key in input,
- count[key] += 1

Construct position:

- Initialise first position as a 1
- position[i] = position[i-1] + count[i-1]

Construct output

- Go through input, looking at each (key, val)
- Set output[position[key]] to the (key, val) pair from input
- Increment position[key]

count

1	1
2	0
3	3
4	0
5	1
6	0
7	2
8	1

position

1	2
2	2
3	5
4	5
5	6
6	6
7	8
8	9

Output

(1,p)	(3,a)	(3,c)	(3,b)	(5,g)	(7,f)	(7,d)	(8,w)
1	2	3	4	5	6	7	8

Analysis of Stable Counting Sort

- We have made count sort:
 - Stable
 - Able to keep associated data
- Time and space complexity?
 - We have not changed the complexity (Though one is faster in practice).
 - Time Complexity is $O(N+U)$
 - Can it be $O(N)$?
 - Yes, when U is $O(N)$ (very efficient)
- Counting sort can also be very inefficient for certain kinds of input, any guesses?
 - Example: $N = 2$, $U = 9223372036854775807$
 - Input: $[0, 9223372036854775807]$
- Using counting sort, can we build a fast way to sort numbers?

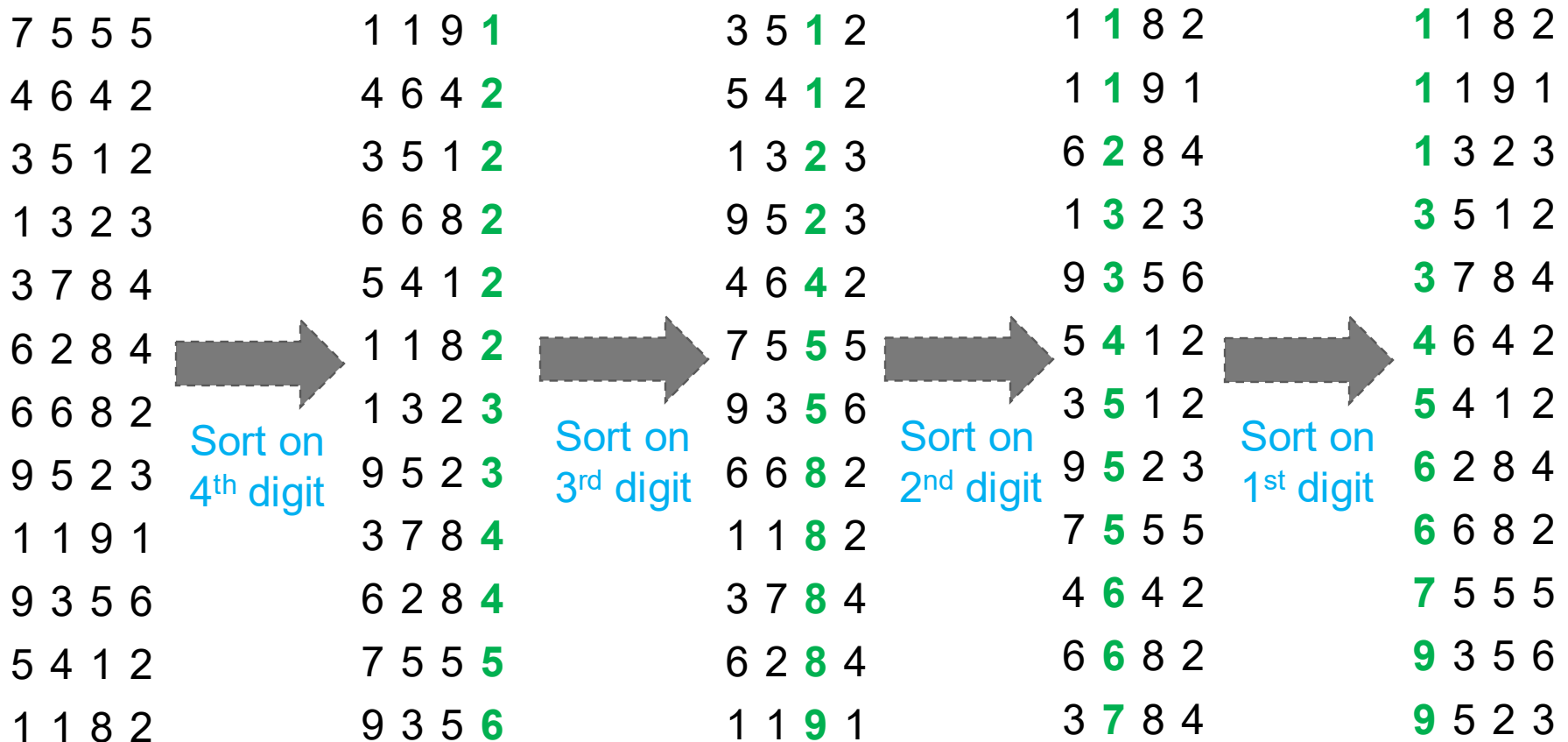
Outline

- Proving correctness of algorithms
- Comparison-based sorting algorithms
 - Selection sort
 - Lower bound for comparison-based sorting
- **Non-comparison sorting algorithms**
 - Counting Sort
 - **Radix Sort**

Radix Sort

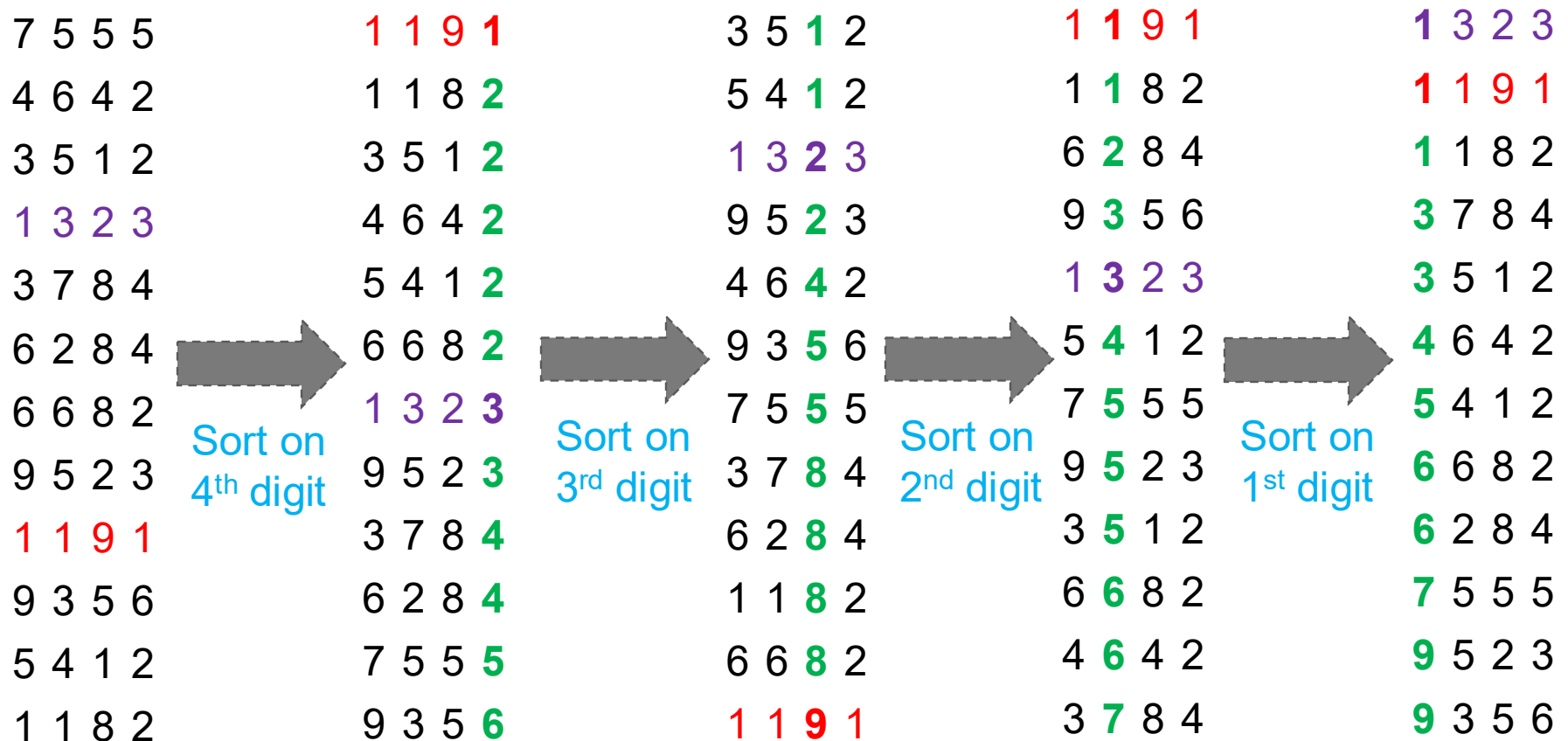
Sort an array of numbers, assuming each number has k digits (why is this often reasonable?)

- Use **stable** sort to sort them on the k-th digit
- Use **stable** sort to sort them on the (k-1)-th digit
- ...
- Use **stable** sort to sort them on 1st digit



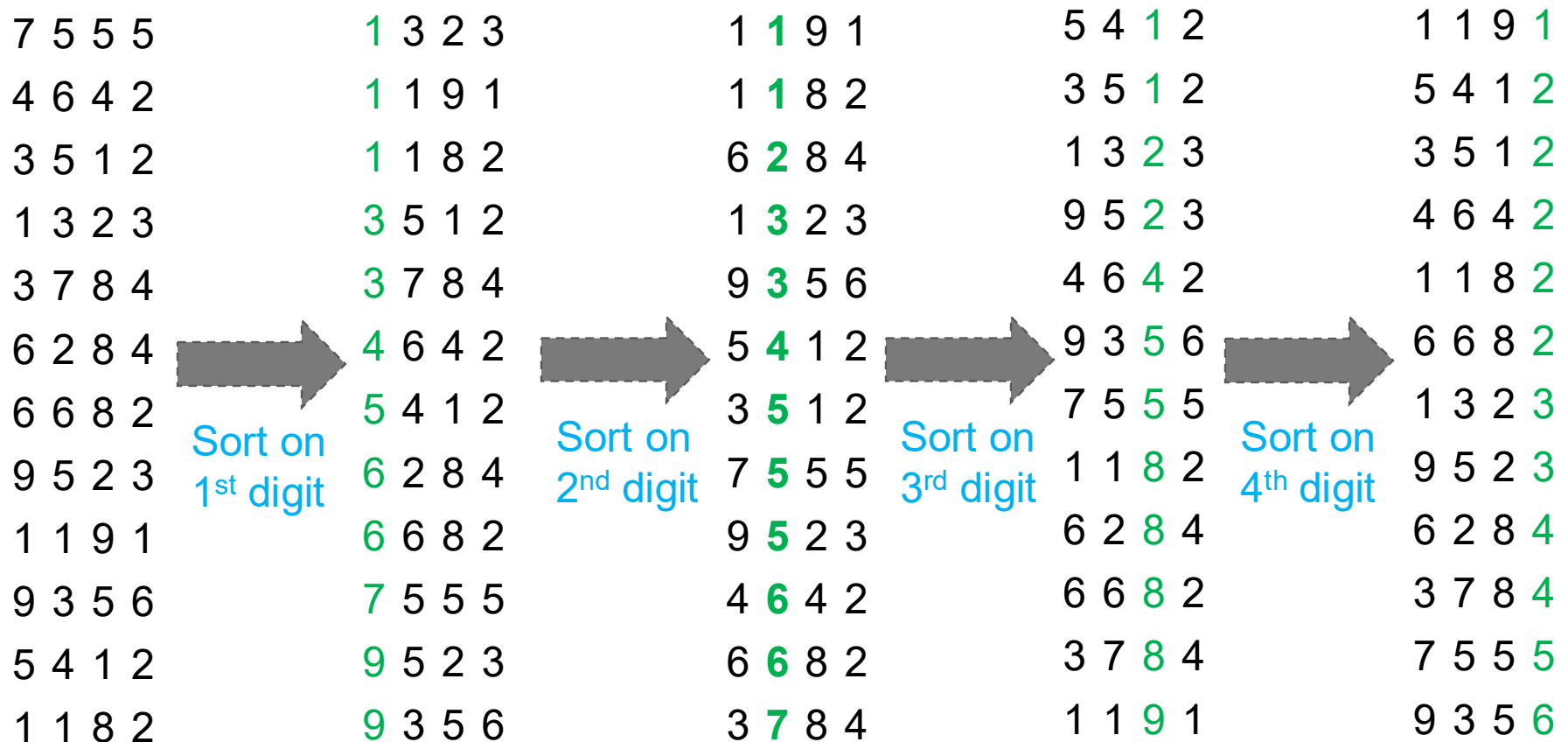
Radix Sort

What happens if we don't use stable sort?



Radix Sort

What happens if we process left to right (or most significant digit to least?)



Analysis of Radix Sort

Sort an array of numbers, assuming each number has k digits

- Use **stable** sort to sort them on the k -th digit
- Use **stable** sort to sort them on the $(k-1)$ -th digit
- ...
- Use **stable** sort to sort them on 1st digit

Assume that N numbers to be sorted and each number has k digits

Assume we are using stable counting sort which has time and space complexity $O(N + U)$

Time complexity of radix sort:

- Number of digits * complexity of counting sort
- $O(k * (N + U)) = O(kN)$ because U is 10

Space complexity of radix sort:

- $O(k * (N + U)) = O(kN)$

Analysis of Radix Sort

But wait!

- Why base 10?
- Variable base, lets call the base “b”
- How many digits does a number have in base b?
- If the number has value M , then it has roughly $\log_b M$ digits
 - The exact number of digits is $\lfloor \log_b M \rfloor + 1$, but since we are doing complexity analysis, we can just say this is $O(\log_b M)$
- So we would need $O(\log_b M)$ count sorts to sort numbers of size M

Analysis of Radix Sort

- So we would need $O(\log_b M)$ count sorts to sort numbers of size M
- What would be the complexity of each counting sort?
- Each counting sort would take $O(N + b)$
- Total cost: $O(\log_b M * (N + b))$
- As we increase $b...$
 - Number of count sorts goes **down**
 - Cost of each count sort goes **up**

Analysis of Radix Sort

- Total cost: $O(\log_b M * (N + b))$

We want to find a good balance ...

- Notice that each count sort will be $O(N)$ as long as the base b is $O(N)$
- As an example, pick $b = N$ (i.e. the base is the number of elements in the input)
- Total cost: $O(\log_N M * N)$
- What would be the value of M which makes the total cost $O(N)$?
- If M is $O(N^c)$, then total cost: $O(\log_N N^c * (N)) = O(CN) = O(N)!!!$
- Of course, practical considerations also matter:
 - Choosing a base which is a power of 2 is probably good
 - Cache/localisation considerations...

Sorting Strings

- How can we apply the count sort/radix sort idea to strings?
- Strings have “digits” which are letters
- The mapping can be done using ASCII:
 - e.g., in python:
 - `ord("A")` gives 65.
 - `ord("B")` gives 66.
 - and so on...
- The radix is now 26 (or however many characters we are using, e.g. 256)

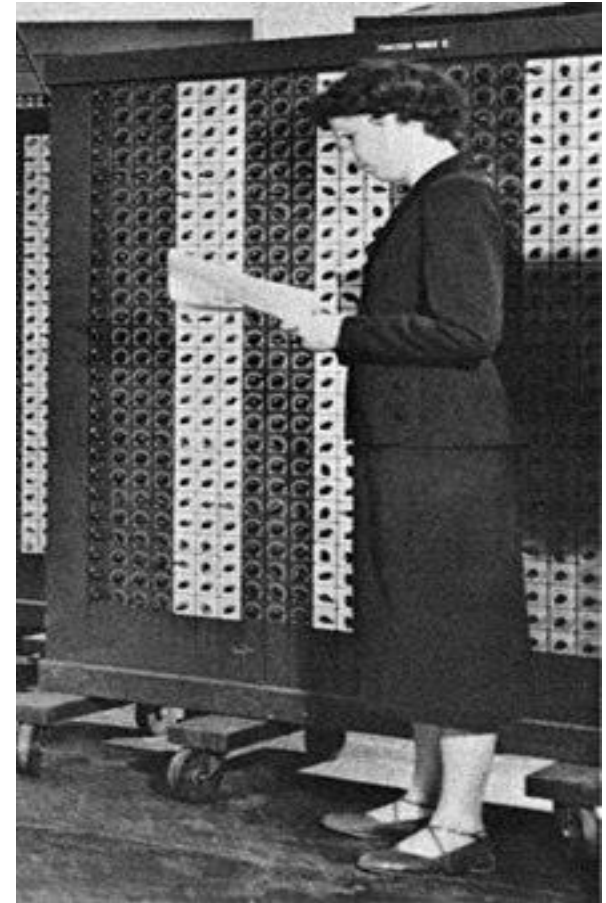


<https://flux.qa/F7CWW9>

Algorithm's Heroes

Betty Holberton (March 7, 1917 – December 8, 2001)

- An American mathematician and programmer
- One of the six original programmers of the first electronic digital computer ENIAC (Electronic Numerical Integrator and Calculator)
- She was the inventor of breakpoints in computer debugging
- Once she said: By 1951 engineers had well accepted the computer, but the business world was still very sceptical about it, because they had such questions as, "How do I know the data is on the magnetic tape?" And "What is the legal implication of data on a magnetic tape?"



Reading

- Course Notes: Sections 1.1, 1.5, 3.1, 3.3 and 3.4
- You can also check algorithms' textbooks for contents related to this lecture, e.g.:
 - CLRS: Section 2.1, Chapter 8
 - Rou: Section 5.6

Concluding Remarks

Summary

- Loop invariants can be used to prove correctness
- Stable sorting, in-place algorithms
- Non-comparison sorting

Coming Up Next

- Quick Sort and its best/worst/average case analysis
- How to improve worst-case complexity of Quick Sort to $O(N \log N)$

Preparation required before next lecture

- Make sure you understand this lecture completely
- Try to implement radix sort yourself